NOTE This option can be changed by setting the property `ConsumerConfig.AUTO_COMMIT_INTERVAL_MS_CONFIG` like this: `.withProperty(ConsumerConfig.AUTO_COMMIT_INTERVAL_MS_CONFIG,[millis here])`. For more information, see http://mng.bz/91x0.

To set auto-commit to `true`, use `ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG` in the consumer's settings as follows:

```
val consumerSettings: ConsumerSettings[String, String] =
      ConsumerSettings(
        config,
        new StringDeserializer(),
        new StringDeserializer())
         .withBootstrapServers("127.0.0.1:9092")
         .withGroupId("group01")
         .withProperty(                                                    ❶
               ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, "true")           ❶
```

❶ Kafka keeps the committed position every 5 seconds (the default).

But here's a problem. Suppose your application reads a message and starts processing it, and then auto-commit kicks in before you're done processing the message. If your application crashes and comes back up, it won't process the last message again. It is considered to be committed. Thus, you can lose messages. Fortunately, you have another option besides `plainSource` with auto-commit: a committable source.

14.1.4 Committable sources